

Строительные Агенты

Создайте своего собственного безголового агента

[Copy page](#) ▼

Создайте «безголового» ИИ-агента для инструментов командной строки, API-серверов и конвейеров автоматизации

ⓘ Хотите создать интерактивный терминальный агент с настраиваемым пользовательским интерфейсом? Ознакомьтесь с руководством [«Создание собственного агента TUI»](#).

Навык [create-headless-agent](#) позволяет создать безголовый агент на TypeScript + Bun — без пользовательского интерфейса терминала, только со структурированным вводом и выводом. Он предназначен для инструментов командной строки, API-серверов, обработчиков очередей и конвейеров автоматизации, где нужен агент, работающий программно.

В основе сгенерированного проекта лежит [@openrouter/agent](#) для внутреннего цикла (вызовов модели, выполнения инструмента, условий остановки) — тот же SDK, который используется в [Agent TUI](#), с неинтерактивным внешним слоем.

Когда стоит создать собственный

Создание безголового агента имеет смысл, когда:

- Вам нужна безголовая автоматизация — пакетная обработка, конвейеры CI, рабочие очереди или структурированная проверка выходных данных
- Вам нужны пользовательские инструменты — ваш агент взаимодействует с вашими собственными API, базами данных или системами, зависящими от домена, к которым не могут получить доступ обычные агенты

- Вы хотите контролировать цикл — вам нужны пользовательские условия остановки, ограничения затрат или логика выбора модели, которые не предоставляются размещенными агентами
- Вы отправляете продукт — агент является частью вашего приложения, а не инструментом разработчика, и вам необходимо владеть точкой входа (CLI, сервером API, встроенной)
- Вам нужен структурированный вывод — потоки событий NDJSON, коды выхода или ответы, проверенные схемой, для программного использования
- Вы хотите научиться — понимание того, как работают агенты на уровне инструментального исполнения, позволяет вам лучше использовать и отлаживать их

Если вам нужен интерактивный терминал, используйте вместо него [навык Agent TUI](#).

Установите навык

Навык `create-headless-agent` входит в коллекцию [OpenRouter Skills](#). Установите его с помощью вашего агента для написания кода на основе ИИ:

[Интерфейс командной строки GitHub](#) Код Клода Курсор

Требуется [GitHub CLI](#) версии 2.90.0+. Работает с Claude Code, Cursor, OpenCode, Codex, Gemini CLI, Windsurf и [многими другими агентами](#):

```
1 gh skill install OpenRouterTeam/skills create-headless-agent
```

После установки попросите своего агента сделать что-то вроде “создать безголового агента” или “создать агента с интерфейсом командной строки”, и навык активируется автоматически.

Предварительные условия

- [Bun](#)
- Ключ [OpenRouter API](#)

Как это работает

Как и навык TUI, «безголовый» навык предоставляет вашему агенту по написанию кода интерактивный контрольный список. Вы выбираете инструменты и модули, и он генерирует полный проект, но вместо REPL точка входа принимает запросы через `--prompt`, позиционные аргументы или стандартный ввод и выводит обычный текст, потоки событий NDJSON или просто код завершения.

Что @openrouter/agent обрабатывает

Беспокойство	Как SDK справляется с этим
Модельные вызовы	<code>client.callModel()</code> — один звонок, любая модель на OpenRouter
Выполнение инструмента	Определите инструменты с помощью <code>tool()</code> и схем Zod; SDK проверит входные данные и вызовет вашу функцию <code>execute</code>
Multi-turn	The SDK loops (call model -> execute tools -> call model) until a stop condition fires
Stop conditions	<code>stepCountIs(n)</code> , <code>maxCost(amount)</code> , <code>hasToolCall(name)</code> , or custom functions
Streaming	<code>result.getTextStream()</code> for text deltas, <code>result.getToolCallsStream()</code> for tool calls
Cost tracking	<code>result.getResponse().usage</code> with input/output token counts
Shared context	Type-safe shared state across tools via <code>sharedContextSchema</code>

Output modes

The generated CLI supports three output modes:

Mode	Flag	Description
Text (default)		Streams text deltas to stdout
JSON	<code>--json</code> / <code>-j</code>	NDJSON event stream — one <code>AgentEvent</code> per line
Quiet	<code>--quiet</code> / <code>-q</code>	No output; exit 0 on success, 1 on error



```
# Text mode (default)
2 bun run src/cli.ts --prompt "List all TypeScript files"
3
4 # JSON event stream for programmatic consumption
5 bun run src/cli.ts --json --prompt "Search for TODO comments"
6
7 # Quiet mode for CI/scripts (exit code only)
8 bun run src/cli.ts --quiet --prompt "Fix the linting errors" && echo "Success"
9
10 # Piped stdin
11 echo "Summarize this file" | bun run src/cli.ts
```

Generated project structure

```
my-headless-agent/
  package.json          @openrouter/agent, zod
  tsconfig.json         ES2022, NodeNext, strict
  .env.example          OPENROUTER_API_KEY=
  src/
    config.ts           Layered config (defaults -> file -> env)
    agent.ts            Core runner with retry + event callbacks
    cli.ts              CLI entry point (args / stdin / piped input)
    tools/
      index.ts          Tool registry + server tools
      file-read.ts      Read files (Bun.file)
      file-write.ts     Write/create files (Bun.write)
      file-edit.ts      Search-and-replace with diff
      glob.ts           Find files by pattern (Bun.Glob)
      grep.ts           Search content by regex
      list-dir.ts       List directory entries
      shell.ts          Execute commands (Bun.spawn)
      fetch-url.ts      Fetch and extract page text
```

Запустите его с помощью:

```
1 export OPENROUTER_API_KEY="sk-or-..."
2 bun run src/cli.ts --prompt "What files are in this directory?"
```

Customization options

The skill presents a checklist when invoked. Items marked `on` are pre-selected defaults.

Server tools

Executed by OpenRouter server-side — zero client code needed.

Tool	Default	Description
Web Search	<code>on</code>	Real-time web search via <code>openrouter:web_search</code>
Web Fetch	<code>on</code>	Fetch and extract page text via <code>openrouter:web_fetch</code>
Datetime	<code>on</code>	Current date/time via <code>openrouter:datetime</code>

Client-side tools

Generated into `src/tools/` with Bun-native implementations.

Tool	Default	Description
File Read	<code>on</code>	Read files with <code>Bun.file</code>
File Write	<code>on</code>	Write/create files with <code>Bun.write</code>
File Edit	<code>on</code>	Search-and-replace with diff output
Glob/Find	<code>on</code>	Find files by pattern with <code>Bun.Glob</code>
Grep/Search	<code>on</code>	Search file contents by regex
Directory List	<code>on</code>	List directory entries
Shell	<code>on</code>	Execute commands with <code>Bun.spawn</code>
Fetch URL	<code>on</code>	Fetch and extract text from URLs

Modules

Optional architectural components for the headless agent.

Module	Default	Description
Session Persistence	on	JSONL append-only conversation log
Retry with Backoff	on	Automatic retry on transient API errors
Context Compaction	off	Summarize older messages when context gets long
System Prompt Composition	off	Build instructions from static + dynamic context files
Tool Permissions / Approval	off	Gate dangerous tools behind confirmation
Structured Event Logging	off	Emit structured events for tool calls and errors
Output Schema Validation	off	Validate agent output against a JSON Schema (Ajv)
Webhook Notifications	off	POST events to an external URL on completion or error

Highlighted features

Safe retry on 429/5xx

The generated `runAgentWithRetry` wrapper retries transient API errors (rate limits, server errors) with exponential backoff — but only if no tool calls have executed yet. Once a mutating tool like `file_write` or `shell` has run, replaying the agent from the initial prompt would double-execute side effects. In that case, retries throw immediately instead of risking repeated mutations.

For mid-run resilience (crash-resume, cross-process approval flows), pair with the optional `Session Persistence` module, which writes every message to a JSONL file so the agent can pick up where it left off.

Structured output with `--output-schema`

Constrain the agent's final response to match a JSON Schema using Ajv. The scaffold is tolerant of markdown fences, so schemas work even when the model wraps JSON in code blocks:

```

1 cat > report.schema.json <<'EOF'
2 {
3   "type": "object",
4   "properties": {
5     "summary": { "type": "string" },
6     "count": { "type": "integer", "minimum": 0 }
7   },
8   "required": ["summary", "count"],
9   "additionalProperties": false
10 }
11 EOF
12
13 bun run src/cli.ts --output-schema report.schema.json \
14   "Analyze README.md and return a JSON report with summary and count fields"

```

Коды выхода:

- 0 — agent succeeded and output matched schema
- 1 — agent or API error
- 2 — output failed schema validation (Ajv error message on stderr, or emitted as a `validation_error` event in `--json` mode)

Entry points

The skill generates a CLI entry point by default, but you can also ask for:

- HTTP server — `Bun.serve()` with SSE streaming for building web-accessible agents
- MCP server — expose the agent as an MCP tool for other agents to call

Resources

- [Create Headless Agent skill README](#)
- [OpenRouter Skills repository](#)
- [@openrouter/agent on npm](#)
- [OpenRouter TypeScript SDK](#)

- [Server Tools documentation](#)
 - [OpenRouter API keys](#)
-

[◀ Build Your Own Agent TUI](#)

[Choose a Video Generation Model ▶](#)
